

Материалы кафедры ММП факультета ВМК МГУ. Введение в глубокое обучение.

Занятие 25. Неявные представления в 3Д

Занятие провёл: Алиев Мишан (mail:
maliev@hse.ru)

Материалы составил: Алиев Мишан (mail:
maliev@hse.ru)

Материалы основаны на лекции
Струминского Кирилла Алексеевича

Москва, Весенний семестр 2026

Источники:

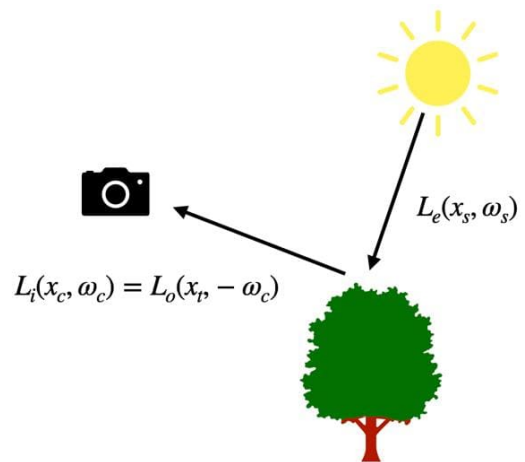
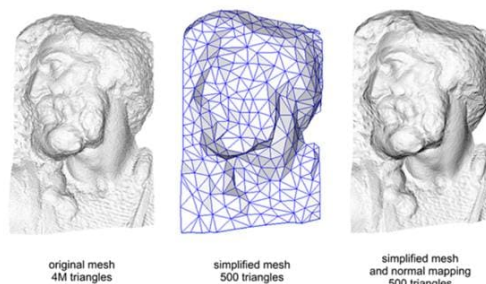
- [Мишан Алиев, Кирилл Струминский](#)
- [Лекция Струминского Кирилла Алексеевича на курсе "3D CV" факультета ФКН](#)
- **DeepSDF**: [Park J.J. et al., 2019](#), примерно 4200 цитирований
- **NeRF**: [Mildenhall B. et al., 2021](#), примерно 11800 цитирований
- **SIREN**: [Sitzmann V. et al., 2020](#), примерно 3000 цитирований
- **Plenoxels**: [Yu A. et al., 2021](#), примерно 200 цитирований.
- **Instant NGP**: [Müller T. et al., 2022](#), примерно 4300 цитирований.
- **pixelNeRF**: [Yu A. et al., 2021](#), примерно 1900 цитирований.
- **LERF**: [Kerr J. et al., 2023](#), примерно 400 цитирований.

- **NerfAcc**: [Li R. et al., 2022](#), примерно 100 цитирований.
- **Mip-NeRF**: [Barron J.T. et al., 2021](#), примерно 2200 цитирований.
- **Mip-NeRF 360**: [Barron J.T. et al., 2022](#), примерно 1900 цитирований.
- **NeRF++**: [Zhang K. et al., 2020](#), примерно 1100 цитирований.
- **Zip-NeRF**: [Barron J.T. et al., 2023](#), примерно 500 цитирований.
- **Camp**: [Park K. et al., 2023](#), примерно 60 цитирований.

Лекция посвящена неявным 3D-представлениям и тому, как они используются для моделирования сцен. Рассматриваются воксельные, нейросетевые и гибридные представления (hash grids, Instant-NGP) как компромисс между качеством, скоростью и памятью. Основной пример – Neural Radiance Fields (NeRF), где сцена задается полем плотности и излучения, а изображение получается интегрированием вдоль лучей. Обсуждаются методы ускорения и улучшения качества: разреживание, иерархическое сэмплирование, anti-aliasing (Mip-NeRF, Mip-NeRF-360, Zip-NeRF). Отдельный блок посвящен улучшению геометрии с помощью SDF, Eikonal loss и последующему извлечению мешей (Marching Cubes)

Preliminaries

Radiance Functions



$$L_o(x, \omega_o) = L_e(x, \omega_o) + \int_{\Omega} f(x, \omega_i, \omega_o) L_i(x, \omega_i) (\omega_i \cdot \mathbf{n}) d\omega_i$$

Попытаемся воспроизвести физический процесс формирования изображения.

Можно говорить о физической симуляции, когда фотоны из источника света летают по сцене и прилетают в камеру, а можно говорить про математический аппарат, который

описывает эту процедуру с помощью непрерывного интегрального уравнения, написанного внизу.

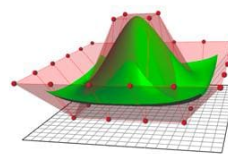
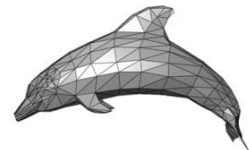
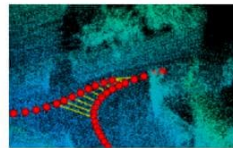
Оно связывает несколько видов функции светимости: L_o – outgoing – исходящая, L_e – emitted – испущенная, L_i – incoming (входящая). Это интегральное уравнение говорит, о том, что сколько света вышло из точки и состоит из двух слагаемых: сколько света сама точка испустила и сколько света было отражено, пришедшего из разных направлений (имеем интеграл по направлениям, из которых пришел свет).

Само уравнение возникать скорее не будет, мы будем говорить про алгоритмы, которые будут стараться эту функцию светимости выучить из сырых данных.

В контексте функций светимости стоит добавить про карты нормалей. Вообще одну и ту же картинку, которую мы видим глазами, мы можем получить несколькими способами. Карты нормалей могут передавать сложную геометрию в рамках простой геометрии. На картинке есть скан древней статуи, то, какой формой она обладает. Дальше запустили алгоритм, который упрощает эту сетку и находит низкополигональную сетку, которую проще хранить и рисовать. Справа же нарисована эта низкополигональная сетка только с картой нормалей у каждого треугольника, которая примерная имитирует реальную сложную поверхность (там более сложная функция светимости, чем у простого треугольника). За счет более сложных зависимостей в функции светимости можно создавать иллюзию более сложной структуры, чем она есть на самом деле.

Common Representations

- Explicit
 - Point clouds
 - Polygonal meshes
 - Parametric $x = g(t)$
- Implicit
 - Isosurfaces $x : f(x) = 0$



Как правило представление трехмерных данных делят на два типа: явное и неявное. Явное позволяют получить точку на поверхности простым и явным способом.

Первый вариант – облака точек, которые представляют сцену набором точек, образующих их. Например, это датчики типа Lidar на беспилотниках, которые

собирают эти данные. Есть проблема, мы не понимаем, где дыры и проблемы в данных, так как поверхности как таковой нет.

Полигональные сетки – объект состоит из треугольников, образованных вершинами сетки. То есть это наборы точек и способы их соединения. Как правило, это треугольники. Такие данные трудно восстанавливать, потому что не оч понятно, как объекта покрывать треугольниками.

Параметрическая сетка – берем значение параметра, вычисляем для значения параметра функцию, описывающую поверхность линии уровня и получаем фигуру. Позволяют представить бесконечно гладкие фигуры. Не так часто встречаются в приложения, так как часто сложную фигуру так параметризовать сложно.

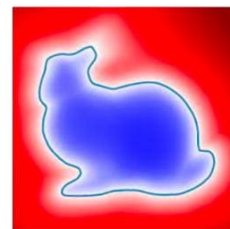
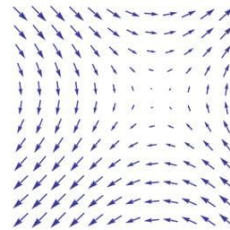
С другой стороны есть неявное представление, где мы не можем просто так взять и посмотреть на точки поверхности объекта. Классическое представление – функция линий уровня, она возвращает значения для каждой точки пространства. Поверхность – множество точек, где у функции какое-то конкретное значение. На основе таких представлений заработала трехмерная реконструкция.

Fields for Volumetric Representations

О неявных представлениях в наше время говорят в терминах полей. Давайте поговорим о них.

Vector Fields

- Used in physics and beyond
- Relevant examples
 - Light absorption coefficients
 - Signed Distance Fields
 - Occupancy Grids
 - Images



<https://en.wikipedia.org/wiki/X-ray>
<https://arxiv.org/abs/1901.05103>
<https://en.wikipedia.org/wiki/Lenna>

Поле – термин физиков для описания взаимодействий и сил. Грубо говоря, это функция, которая точке пространства сопоставляет либо вектор, либо число. Например, в физике сила земного притяжения описывается неким полем сил, где в каждой точке пишем направление притяжения.

Это довольно удобный инструмент, которым можно описывать много разных эффектов, в том числе и фигуры: можно записать поле коэффициентов, которое каждой точке пространства будет сопоставлять, сколько точка поглощает свет. Картинка с вином: просветили рентгеновскими лучами сцену насквозь, стекло поглощает света больше, поэтому имеем такое представление. если говорить про фигуры, то можно говорить по **SDF = signed distant field**. Фигуру можно задать как линии уровня, то есть сказать, что есть какая-то функция, которая равна 0 на поверхности этой фигуры, также в каждой точке пространства записано расстояние до ближайшей точки на поверхности описываемой фигуры. Оссирансу grid – поля занятости – 1, где фигура плотная, 0, где фигуры нет. Поле может описывать изображение – выдаем три числа в каждой точке картинки, формируя изображение.

Volumetric Representations

- Surface is a common abstraction in computer graphics
 - Convenience, hardware optimization
 - Sometimes fail to represent real world phenomena
- As an alternative, we will represent scenes with “density” functions
 - Is there something in this point of space?
 - Does light pass through this point in space?
- **Scalar field:** each spatial point stores a scalar value



https://www.reddit.com/r/MicrosoftFlightSim/comments/ymz0ex/is_this_really_what_photogrammetry_looks_like_for/

Почему нельзя описать все полигональными сетками? Фрактальные структуры или с мелкой детализацией потребуют много ресурсов, чтобы их описать.

Скриншот из игры Microsoft Flight Simulator, где все представлено полигонами. Проблема в деревьях, листочках.

Объемным представлениями проще. Фрактальные структуры можно описать проще просто математикой, так как задаем линии уровня.

Также проще задавать объемные фигуры, например, облака, где не очень понятна, где поверхность (плотен в центре, непонятны границы): можем просто задавать поглощение света в разных местах (в центре побольше, на краях поменьше).

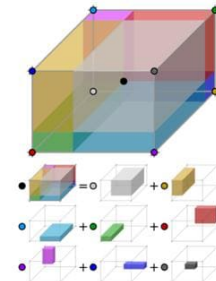
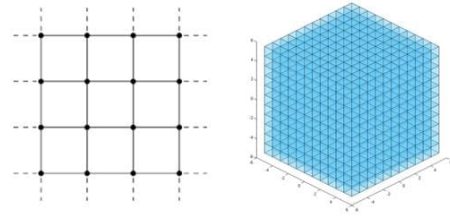
Еще пример – волосы, их тоже представляют объемным представлением.

Grid Representations

Pixels, Voxels, etc.

- Consider a field $f: [0,1]^3 \rightarrow \mathbb{R}^c$
- Define a $h \times w \times l$ grid in $[0,1]^3$
- Parameterize field with a tensor $G \in \mathbb{R}^{h \times w \times l \times c}$
- Interpolate G to define the field on $[0,1]^3$

$$f(x) = \sum_{i,j,k} G_{ijk} K(x, x_{ijk})$$



https://en.wikipedia.org/wiki/Trilinear_interpolation

Как параметризовать/задать поля? Самое наивное – задать единичный куб, в котором находится сцена. Дальше побить на маленькие кубики, завести решетку и в каждом узле решетки вписать число, которое будет говорить, насколько плотно значение поля в вершине этого кубика.

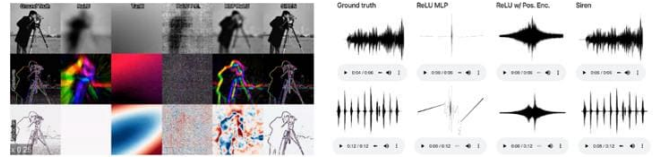
Задается классическим тензором размера (h, w, l). Также надо ввести один доп размер – количество каналов представления (скалярное поле – c=1, цвета – c=3).

Если теперь хотим поле на всем объеме – описываем значение поля в пространстве линейной интерполяцией – с какими-то весами складывать значения в узлах маленьких решеток (веса зависят от расстояния точки до узлов).

Проблема – если хотим детальную сцену, нужно много узлов, что требует много памяти, хотя все считается быстро и точно. Чисто сеточное представление не так популярно в графике.

Neural Fields

- Takes a point x , returns a vector
- Parameterized MLP $f(\cdot)$
- Input encodings $\gamma(x)$ / trigonometric activations help
$$\gamma(x) = [\sin(2^k x), \cos(2^k x)]_{k=1}^h$$
- Applies to various domains such as photo, video, audio & 3D data



[Sitzmann](#) V. et al. Implicit neural representations with periodic activation functions //Advances in Neural Information Processing Systems. – 2020. – Т. 33. – С. 7462-7473.
[Mildenhall](#) B. et al. Nerf: Representing scenes as neural radiance fields for view synthesis //Communications of the ACM. – 2021. – Т. 65. – №. 1. – С. 99-106.

Можно найти более экономное представление для полей, используя DL. Здесь нейронка – алгоритм сжатия данных. Она, например, по координате пикселя картинки будет выдавать ее цвет.

Вообще можно так зрнить данные различных доменов. Например, видео. У нас есть координата на экране и момент времени, можем пытаться возвращать цвет пикселя в этот момент времени в этой точке экрана. Странное решение, но так можно.

Достаточно обычной [MLP](#). Обучается с помощью обычного градиентного спуска.

Важно еще использовать специальные позиционные энкодинги для координат. Практика показывает, что с ним учится быстрее и лучше. Они такие же, как и в трансформерах. На выходе те же $[0,1]$, но если есть небольшое изменение координаты, то обычная сеть даст небольшое изменение выхода (гладкое поведение), а такой энкодинг сильно изменит $\sin()$ и $\cos()$, что является хорошим признаком резких переходов для модели. С ними учится лучше.

На слайде [пример](#), где авторы предложили обычные активации заменить на синусы и косинусы, получается хорошее представление. Обычно конкатенируется x с этими энкодингами.

Дешевле по памяти, чем решетки. Несколько мегабайт. Правда не всегда контролируем потерю информации сцены, в отличие от классических способов сжатия данных, как JPEG. Еще минусы – декодирование дорогое, так как каждый раз надо вызывать много раз нейронку.

Hybrid Representations

Pros and cons of the above solutions

Is there a solution compromising between the two?

MLP

- Size: ~5mb
- Slow
- Trade-off **speed** for fidelity

Voxel Grid

- Size: ~1Gb
- Fast
- Trade-off **memory** for fidelity

Fridovich-Kail S. et al. [Planoxels](#): Radiance fields without neural networks // CVPR 2022
Müller T. et al. Instant neural graphics primitives with a [multiresolution](#) hash encoding // ACM Transactions on Graphics (ToG). – 2022. – T. 41. – №. 4. – С. 1-15.

Hybrid Representations

Voxel Based

- Parameterize a grid with $G \in \mathbb{R}^{h \times w \times l \times c}$
- For an input point x get the representation with interpolation

$$g(x) = \sum_{i,j,k} G_{ijk} K(x, x_{ijk})$$

- Define an MLP $h(y)$ to map the representation to field vectors

$$f(x) := h(g(x))$$

- Middle ground between MLPs and voxel grids

Не хотим хранить сетку высокого разрешения, но MLP тоже не хочется. Будем искать решение посередине.

Параметризуем наше поле не скалярными значениями, а C числами, которые будут эмбедингами данной точки пространства. А дальше в точке пространства будем считать значение эмбедингов, интерполируемое из ближайших узлов, а потом завести доп сеть, которая по эмбедингам в окрестности будет возвращать значение поля. Сетка берется чуть менее точного разрешения, так как часть инфы может додумать MLP по эмбедингам.

Эмбединг – это числа, зависящие от координат пространства, в них больше инфы, поэтому и сеть может быть меньше, так как она только обрабатывает инфу, а не хранит ее в себе.

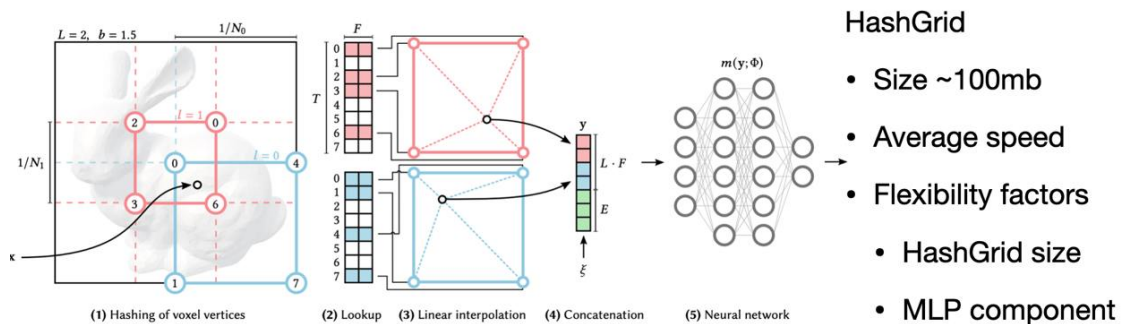
Все дифференцируемо и настраивается. Меньше памяти и быстрее работает.

Значения в узлах мы выучиваем. Можно в сеть, которая обрабатывает эмбединги, можно подавать доп инфу, например, нам нужен цвет в зависимости от угла обзора, откуда мы смотрим.

Часто бывает много пустых мест и мест, которые нас не интересуют, на которые тратим ресурсы. Сетка в таких случаях – проигрышный вариант, так она покрывает весь объем.

Instant-NGP

Saving time and memory with HashGrids



Müller T. et al. Instant neural graphics primitives with a multiresolution hash encoding //ACM Transactions on Graphics (ToG). – 2022. – T. 41. – №. 4. – С. 1-15.
<https://github.com/NVlabs/instant-ngp>

Instant-NGP – это сейчас стандарт представления полей, собирающий все вышесказанное.

Давайте вместо хранения значения представления в узлах решетки заведем хэш-функцию.

Вот хотим в узле решетки посчитать значения представлений, вместо явного хранения в узлах значений, посчитаем хэш индекса решетки и пойдём в хэш-таблицу по значению хэш-функции. В более маленькой хэш-таблице будем хранить эмбединги.

Когда будем вычислять значения через линейную интерполяцию, к нам в данный индекс через хэш-таблицу с векторами будут приходить из разных точек пространства. Векторы будут зраниять в себе много информации, описывая много точек. Обновляем веса нейронки и значения в хэш-таблицах.

Есть несколько уровней масштаба: голубой и розовый (хэш-таблиц столько же). Вот у нас есть точка, смотрим на голубой квадрат. У него рядом 4 узла, берем координаты, считаем хэши, идем хэш-таблицу, берем значения и делаем интерполяцию. Разные точки пространства приходят в одну точку хэш-таблицы, тем самым взяв маленькую хэш-таблицу, мы разряжаем пространство.

Требуется меньше памяти, так как храним только хэш-таблицу меньшую по размеру.

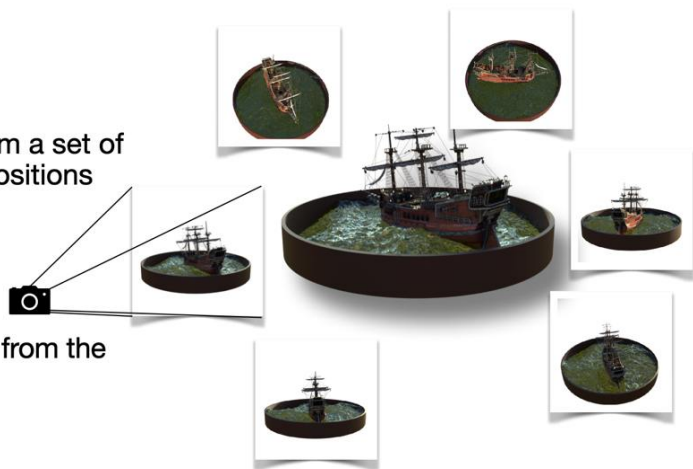
Еще плюс, что хэши быстро считаются на видеокартах и это работает оч быстро из-за хорошей оптимизации под современные архитектуры видеокарт. Еще те же чуваки сделали `tiny-cuda-nn`, которая в пониженной точности на `cuda` реализует примитивные нейросети и эмбеддинги, которую можно юзать.

Novel View Synthesis with Neural Radiance Fields

Поговорим, откуда эти самые Neural Radiance Fields поля можно брать. Их получают, решая задачу Novel View Synthesis.

Novel View Synthesis

- Problem setup:
 - A set of pictures taken from a set of pre-determined camera positions
 - Test camera position
- Goal:
 - A picture of a scene taken from the test camera position



[Mildenhall B. et al. Nerf: Representing scenes as neural radiance fields for view synthesis //Communications of the ACM. – 2021. – Т. 65. – №. 1. – С. 99-106.](#)

Четкого перевода Novel View Synthesis нет, поэтому буду говорить по-английски.

Что дано? Есть набор изображений, у которых есть ракурсы, с которых они сняты, и внутренние параметры камеры, то есть процесс формирования изображений нам известен.

Цель научиться генерировать изображения сцены с новых ракурсов, как будто это камера реально стояла на сцене и сделала снимок.

Критерий качества решения задачи: на отложенной валидационной выборке проверяем сгенерированные виды.

Ничего заранее про геометрию, структуру и т.п. нам неизвестно, просто хотим научиться генерировать сцену с новых ракурсов.

Where Does the Data Come From?

- Structure From Motion (COLMAP)
 - Takes unposed images as input
- ARKit
 - Records device trajectory along with images



Откуда в такой постановке задачи брать данные?

Часто под ковер спрятан COLMAP. Его используют для предобработки данных. Часто нам поступает на вход набор снимков статичной сцены, снятых с разных ракурсов, подаем их в COLMAP, который примерно прикидывает, как были расположены камеры и какие их внутренние параметры. А дальше можем переходить к решению поставленной задачи.

Другой вариант – ARKit – библиотека от Apple, которая позволяет на основе данных с гироскопа и акселерометра с телефона и того, что снято на видео, примерно прикинуть, где телефон был в пространстве во время съемки. Но проблема в том, что датчики шумные и траектории получаются неточные и требуют доработки.

Глобально два варианта – COLMAP или датчики поверх видео.

Neural Radiance Fields

Representing a Scene with Neural Fields

- Represent a scene with two fields
 - Density: $\sigma(x) : \mathbb{R}^3 \rightarrow \mathbb{R}^+$
 - Radiance: $C(x, d) : \mathbb{R}^3 \times S^2 \rightarrow \mathbb{R}^3$
- Density represents is related to opacity
- Radiance represents color of a point
- Architecture: MLP + positional embeddings



[Mildenhall](#) B. et al. Nerf: Representing scenes as neural radiance fields for view synthesis //Communications of the ACM, – 2021. – Т. 65. – №. 1. – С. 99-106.

Поговорим про поля, которые нам понадобятся, в таком решении, как Neural Radiance Fields.

Для описания сцены понадобятся две вещи: Геометрия – будем описывать ее с помощью поля плотности. Будем говорить, насколько точка непрозрачна, то есть долю света, которая пропускает данная точка. Поле светимости – в каждой точке пространства хотим задать, сколько от нее исходит света в направлении d . Направлении важно, так как многие объекты в реальном мире меняют цвет, когда мы смотрим на них под разным углом.

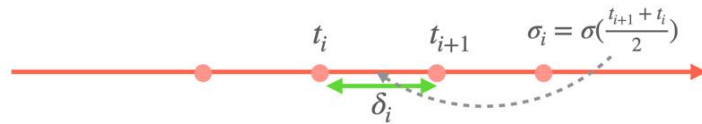
Есть иллюстрация, где при движении меняется цвет зеркального шара (угол отражения равен углу преломления).

Все варианты архитектур, описанных сегодня, применимы. Исторически, в первой статье была MLP + positional embeddings.

Computing Pixel Color

- Density $\sigma(t) \in [0, +\infty)$ is related to opacity at t
- Divide the ray with points $t_1, \dots, t_n$ and define

$$\alpha_i = 1 - \exp(-\sigma_i \delta_i)$$



- Expected color along a ray is given by

$$C = \sum_i c(t_i) \cdot \alpha_i \prod_{j < i} (1 - \alpha_j)$$

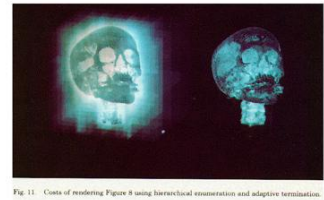
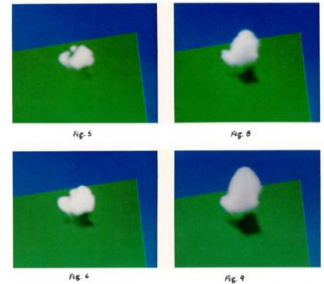


Fig. 11 Costs of rendering Figure 8 using hierarchical enumeration and adaptive termination.



Описали постановку задачи, представление сцены, как рисовать картинку, если есть хорошее начальное приближение картинки.

Алгоритм отрисовки – это ключевой элемент успеха подхода, так как он очень хорош для современного DL и градиентных подходов.

Исторически этот алгоритм возник в литературе по графике, где пытались моделировать объемный эффект в виде облаков или делать визуализацию трехмерных данных (например, МРТ-снимки).

Каждый цвет получен из того, что мы встретим на луче, пока летит в пространстве. Зафиксируем пиксель и из координаты камеры в направлении пикселя выпустим луч.

Чтобы понять, что на луче находится, возьмем его и разобьем на большое число маленьких участков и в каждом посчитаем плотность и цвет.

Плотность будет говорить о том, есть в данном участке что-то или нет. Есть она 0, то ничего нет. Если большие значения, значит что-то плотное и не пропускающее цвет.

Важно, что если на луче несколько плотных точек, то мы будем видеть тот, что ближе к камере, то есть порядок по глубине надо учитывать. Алгоритм, учитывающий этот порядок в дискретном случае называется alpha compositing и он складывает цвет на луче по приведенной ниже формуле.

Поле сигма отображается в альфу, которую встречаем в альфа канале изображений. Альфа принимает значения от 0 до 1. Если сигма большая => альфа = 1, что-то непрозрачное. Если сигма маленькая => альфа = 0, что-то прозрачное.

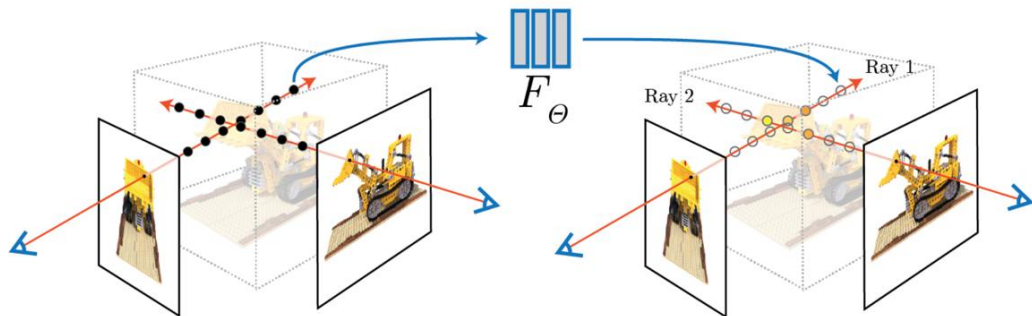
Дальше эти альфы собираются в формулу снизу. Явно учитывает порядок обхода луча. Важно, что веса цветов учитывают не только прозрачность текущей точки, но

точек до нее на луче.

За этим алгоритмом стоит физический процесс распространения физического света в пространстве, дискретный случай которого представлен на слайде.

Все дифференцируемо, градиенты хорошие и все сходится.

Computing Pixel Color



Neural Radiance Fields

Optimization

- Training: minimize the mean squared error $\mathbb{E}_D \|C - C_{gt}\|^2$

$$C = \sum_i c_i \alpha_i \prod_{j < i} (1 - \alpha_j)$$

- Optimize w.r.t. parameters of radiance C and density σ

Традиционно задача Novel View Synthesis решается минимизацией квадрата разницы между картинкой из train и картинкой, которые мы нарисовали по текущему полю плотности и светимости.

NeRF pros and cons

- + Simplicity & Flexibility
- + Photorealism
- + Compression (5mb / scene)
- Ill-posed problem
- Long training time (48 h. / scene)
- Slow rendering (1 min. / frame)



Гибкость – исходную постановку можно модифицировать для других задач.

Данных обычно используется больше при обучении, чем вес модели.

Постановка задачи не совсем корректная: это два кота друг напротив друга или кот напротив зеркала. Можно нам понять по косвенным причинам, а как модели понять? Не все так однозначно, как говорится.

Дальше поговорим про улучшения и применения представленной модели.

NeRF in The Wild & Block NeRF

- Radiance field handle embeddings capturing variability in lightning conditions
- Approach is scalable to large scene reconstruction



<https://nerf-w.github.io/> <https://waymo.com/research/block-nerf/>

Поле светимости выучивается нейронкой. Можно это использовать.

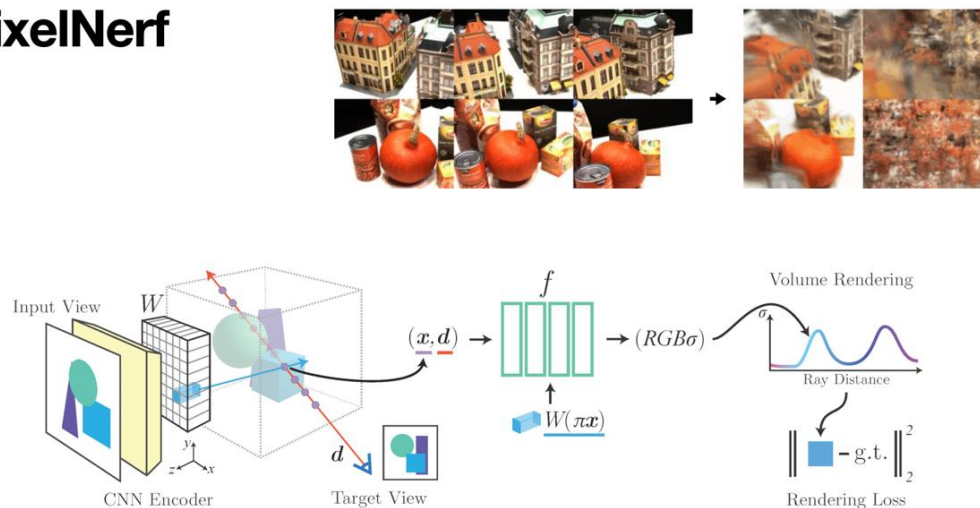
Сцены не всегда статичны, может меняться освещение в реальных данных. Давайте в модель с каждым кадром обучающей выборки подавать эмбеддинг кадра,

кодирующего время дня (по факту освещение кадра), так получим поле светимости, также зависящие от доп параметров освещения сцены для данного момента времени (эти эмбединги можно выучивать). Геометрию же сцены можно представлять с помощью поля, которое не зависит от времени.

NeRF in the wild. Берем кучу разных кадров из интернета и пытаемся выучить модель фонтана Треви (Рим), но с учетом разные снимки получены в разное время при разном освещении. Получилась динамичная сцена, которая может менять освещение.

Block NeRF. Та же самая идея, но практичнее – это реконструкция города Сан-Франциско ребятами из Waymo (беспилотники). Данные из google street view. Одной сеткой не представишь, там хитрее устроено. Можно использовать для обучения беспилотников – имеем фотореалистичные данные, покрывающие все многообразие освещений.

PixelNerf



<https://alexgy.net/pixelnerf/>

У нас в задаче одно поле – это одна сеть, которую мы выучиваем. Немного экзотично для ДЛ. Возникает вопрос, а можно выучить одну сеть, которая будет рисовать все, что мы захотим в 3Д?

Ответили на вопрос в Berkley. Давайте решать задачу во few-shot режиме, то есть на вход будет приходить несколько кадров, а не сотни. Будем учить мета-сеть, которая по трем кадрам будет рисовать реконструкцию сцены. Делать это будем с помощью энкодера, который будем учить.

Будем изображения кодировать с помощью какой-то backbone сети (например, предобученный resnet). Теперь в запускем луче плотность и цвет будем обуславливать еще на эмбединги точки пространства, исходя из представления на основе тех поданных картинок в условии.

Будем проецировать точку на луче на картинку данную, посчитаем на проекции эмбединг и его используем в MLP. Таким образом, реконструируя каждую сцену будем считать эмбединги по сложному геометрическому правилу, объединять их и подавать в одну общую глобальную сеть для всех сцен, тем самым реконструируя нужную нам сцену.

Хорошо обобщается, но в оригинальной статье пробовали на ограниченном наборе данных.

Language Embedded Radiance Fields

- One can replace MSE reconstruction loss with fancier training signals
- CLIP-embeddings for text retrieval and 3D segmentation



Kerr J. et al. LERF: Language Embedded Radiance Fields //arXiv preprint arXiv:2303.09553. – 2023.

У нас в принципе могут быть не только поля светимости, могут быть вообще любые поля.

Как пример, эта [работа](#) предлагает выучить поля из clip представлений картинок. Объяснить CLIP.

Получаем, что если представления вещи и ее описания будут похожи между собой. Считаем CLIP-представления для маленьких кусочков сцены и потом использовать их взамен цвета. В общем можно выучивать поля CLIP-эмбедингов и для таких полей брать текст того, что мы хотим найти на сцене и считать скалярное произведение пикселей с эмбедингом текста и получать тепловую карту с нужной вещью. Работает для 3хмерных сцен. Робот теперь может принести стакан воды.

Improving Base Model: Time

Хоть модель и хороша, она очень медленно работает. Учится сутки, рисует кадр за минуту ... Обсудим то, как улучшить модели.

Давайте поговорим о том, как улучшить скорость работы модели.

Computational Overhead

$$\hat{C} = \sum_i c_i \alpha_i \prod_{j < i} (1 - \alpha_j)$$

- Picture contains $\approx 10^6$ pixels with 10^2 field evaluations per pixel
- Modern GPUs deliver poor frame rate at this cost
- How can we speed up rendering?



Алгоритм рисовки – сумма с большим числом слагаемых, для каждого надо вызвать нейронку.

Сотни миллионов вычислений, что медленно даже с учетом современного железа.

Sparsification

- Each terms in the sum requires running an MLP

$$C = \sum_i c_i \alpha_i \prod_{j < i} (1 - \alpha_j)$$

- If $\sigma(x_i) = 0$, then $1 - \alpha_i = 0$, we can omit i -th term
- Idea: cache $\sigma(\cdot)$, omit i if we know $\sigma(x_i) = 0$
- Outcome: training and rendering 10-30 times faster

[Hedman P. et al. Baking neural radiance fields for real-time view synthesis //Proceedings of the IEEE/CVF International Conference on Computer Vision. – 2021. – С. 5875-5884.](#)
Sun C., Sun M., Chen H. T. Direct voxel grid optimization: Super-fast convergence for radiance fields reconstruction //Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. – 2022. – С. 5459-5469.
[Erilovitch-Kail S. et al. Disvoxels: Radiance fields without neural networks //Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. – 2022. – С. 5501-5510.](#)
Li R., [Tancik M.](#), Kanazawa A. NerfAcc: A General NeRF Acceleration Toolbox //arXiv preprint arXiv:2210.04847. – 2022.

Давайте вернемся в сетап одной сцены – одной модели.

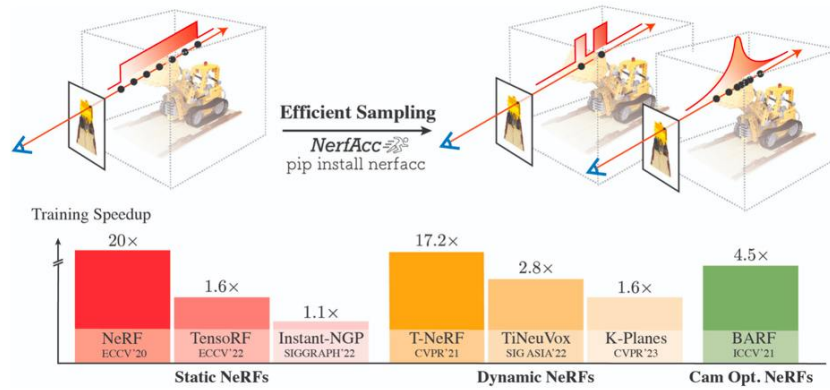
Устоявшееся решение – пропускать пустые участки пространства (например, с помощью булевой сетки) и если встретили что-то непрозрачное, дальше можно не считать.

Объем во время обучения формируется равномерно, поэтому мы довольно рано начинаем понимать, где чего есть или нет.

Ускорение на порядок.

Идея есть в большом числе работ, на слайде приведены часть этих работ.

NerfAcc 



Li R. et al. NerfAcc: Efficient Sampling Accelerates NeRFs // ICCV – 2023

Венцом всех ускорений стала либо [NerfAcc](#), которая идеи с разреживаниями реализует в хорошей абстракции.

Базовый нерф ускоряют в 20 раз.

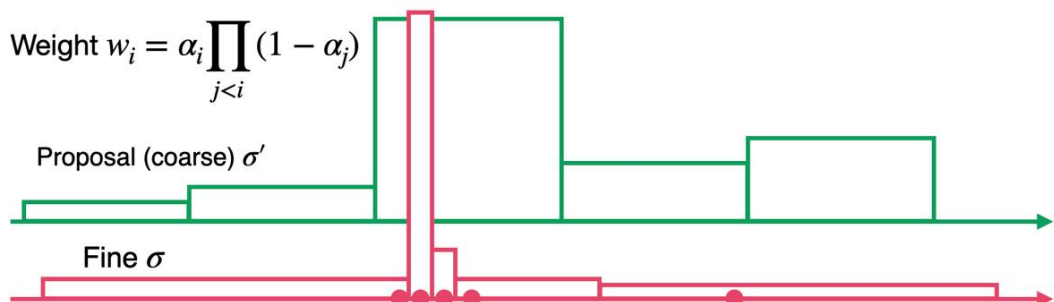
Improving Base Model: Fidelity

Дальше поговорим о качестве.

Hierarchical Sampling

- Until now, we picked m knots uniformly on the ray
- Hierarchical approach picks the knots adaptively

- Weight $w_i = \alpha_i \prod_{j < i} (1 - \alpha_j)$



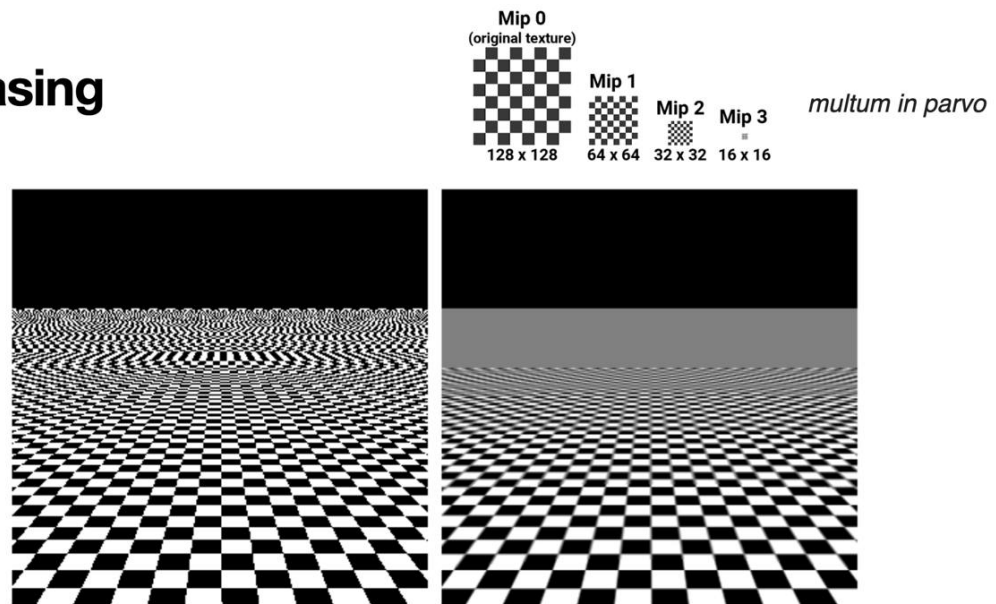
Чтобы точно покрыть сцену нам нужно много точек на луче.

Еще в оригинальной статье придумали иерархическое сэмплирование точек на луче. Наивно – равномерно покрыть луч и считать значения в каждой точке.

Идея: заведем маленькую сетку плотности разреженную, которая будем говорить, есть ли что-то в общих чертах или нет и сэмплировать точки в зависимости от весов этой сетки (веса w). Есть что-то – больше узлов, нет – меньше узлов.

Точнее покрываем луч, да еще и экономнее.

Aliasing



Aliasing – это артефакт дискретизации, возникающий при представлении непрерывных сигналов (изображений, звука и т. д.) в дискретной форме.

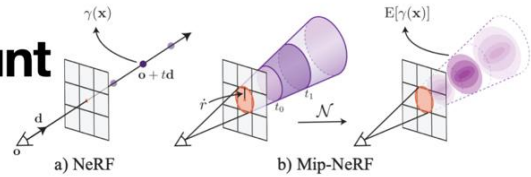
В компьютерной графике алиасинг проявляется в виде "зазубренных" краев, мерцания и других искажений при рендеринге изображений с высоким уровнем детализации.

В контексте NeRF проблема алиасинга возникает из-за того, что пиксели камеры на самом деле покрывают не одну точку в пространстве, а некоторую область (конус). рядом с камерой он покрывает большие участки сцены, а в 10 метрах от камеры, пиксель покрывает пропорционально больший участок сцены.

Однако стандартный NeRF делает выборку света только в отдельных точках вдоль лучей, игнорируя тот факт, что пиксель охватывает большее пространство. Это приводит к потере информации и появлению артефактов. То есть если наш луч улетит далеко и будет считать цвет на дальних участках, он посчитает нерепрезентативный, так как надо учитывать цвет всего участка, а не только его цвета.

Taking Scale into Account

A Parallel Line of Research



- Original NeRF experiments had biased data
 - Camera distances did not change by a significant amount
- Pixels cover a cone-like region in space
- Ideal solution - compute the average radiance over the whole cone
- Mip-NeRF approximates the average radiance/density over a cone segment
- Idea: compute embedding average rather than output average

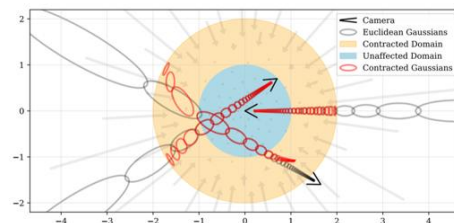
Barron J. T. et al. Mip-nerf: A multi-scale representation for anti-aliasing neural radiance fields //Proceedings of the IEEE/CVF International Conference on Computer Vision. – 2021. – С. 5855-5864.

В отличие от обычного NeRF, который обрабатывает каждый пиксель как тонкий луч и берет выборки точек вдоль этого луча, Mip-NeRF рассматривает пиксель как конусовидный объем в пространстве, расширяющийся с увеличением расстояния. Вместо обработки отдельных точек, Mip-NeRF математически представляет весь объем как 3D гауссово распределение и вычисляет для него интегрированное позиционное кодирование – единый эмбединг, содержащий информацию обо всем объеме сразу. Это позволяет корректно учитывать масштаб и избегать артефактов алиасинга при изменении расстояния до объектов, что невозможно в обычном NeRF, где каждая точка рассматривается изолированно, без учета её окружения.

Этот нюанс критичен для работы с реальными данными, а не синтетическими.

Unbounded Scenes

- Original NeRF experiments had biased data
 - All the training scenes were bounded
- NeRF++ models foreground and background with two networks
- Mip-NeRF-360 maps the space into a ball
 - Does not transform scene center
 - The rest is mapped onto a shell



Zhang K. et al. Nerf++: Analyzing and improving neural radiance fields //arXiv preprint arXiv:2010.07492. – 2020.
Barron J. T. et al. Mip-nerf 360: Unbounded anti-aliased neural radiance fields //Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. – 2022. – С. 5470-5479.

Еще один момент, который не учитывается в ванильных нерфах, что непонятно, где реальные сцены заканчиваются.

Мы пускаем луч, а где его останавливать? На сценах есть гиперпараметр метода, который говорит, что пора остановиться.

Вобщем взоры нейронки нормированы, от 0 до 1. Подавать координаты огромные чревато nan'ами.

Оригинальные эксперименты с NeRF были ограничены фиксированными пространствами, что создавало предвзятость в данных и затрудняло моделирование неограниченных сцен. Для решения этой проблемы **NeRF++** использует две сети для отдельного моделирования переднего и заднего планов, а **Mip-NeRF-360** применяет метод отображения пространства в сферу: центр сцены остается неизменным, а остальная часть сжимается и проецируется на оболочку, что позволяет корректно обрабатывать большие и бесконечные сцены без артефактов.

Zip-NeRF

- Merges two research branches
 - Speed: Instant-NGP
 - Fidelity: Mip-NeRF-360
- Mip-NeRF is designed for MLPs
- Zip-NeRF adapts Mip-maps to Instant-NGP



Barron J. T. et al. Zip-NeRF: Anti-Aliased Grid-Based Neural Radiance Fields //arXiv preprint arXiv:2304.06706. – 2023.
<https://jonbarron.info/zipnerf/>

Апогея всех идей – **Zip-NeRF**. Быстрое и гибкое представление. Сцена нарисована, но можно понять, что это генерация по косякам в отражениях.

Fine-tuning Camera Positions



<https://camp-nerf.github.io/>

Еще есть работа [CampP](#). Позиции камеры COLMAP восстанавливает неидеально, датчики движения тоже неидеальны.

Алгоритм рисовки дифференцируем не только по плотности и цвету, но и по параметрам камеры – их также можно донастраивать с помощью градиентного спуска.

Изменения камеры по разным координатам могут быть разного масштаба, мы пытаемся все свести к одному.